



PAWSITIVE LIFW – FINAL REPORT

Or Avichzer Elmalih & Noa Danon & Mark Tsirlin



20 JUNE 2025

AFEKA
Department of Computer Science

Abstract

PAWsitive Life is a comprehensive Android-based mobile application developed to enhance the everyday experience of dog ownership through a smart, personalized, and location-aware platform. The application enables users to manage their dogs' profiles, schedule and receive veterinary care reminders, and access custom-tailored articles based on each dog's breed, age, and health history.

To support these features, the system integrates Firebase for real-time data storage and user authentication, MapBox API for displaying nearby dog parks, veterinarians, and pet stores, and the Cohere API for generating relevant educational content. Together, these components create an intelligent, responsive ecosystem that adapts to the needs of each user and pet.

Throughout the project, we successfully implemented core modules including dog profile creation, interactive calendar reminders, location-based recommendations for dog parks, veterinarians, and pet stores, and personalized article filtering. The system underwent structured testing across 20 functional test cases, with all tested components performing as expected. While the core functionality is stable and provides a solid foundation, several features remain in early stages and present opportunities for further refinement and enhancement before a full-scale release.

Table of Contents

- 1. Introduction**
- 2. Background and Related Work**
- 3. System Design / Methods / Approach**
- 4. Methodology**
- 5. Implementation**
- 6. Experiments and Results**
- 7. Discussion**
- 8. Conclusion & Future Work**
- 9. References**

Introduction

Motivation

Many dog owners find it increasingly challenging to manage the daily responsibilities associated with pet care. Tasks such as keeping track of vaccinations, medical treatments, grooming, behavioral training, nutrition, and timely social interactions often become overwhelming—especially for first-time owners or those managing multiple pets. The required information is typically scattered across physical notes, calendar apps, veterinary documents, and various websites. This fragmentation leads to inefficiencies, forgotten appointments, or even overlooked health concerns. As dog ownership rates continue to climb globally, so does the need for a centralized, intelligent, and user-friendly platform that brings together all essential aspects of dog care. Our app, PAWsitive Life, was born out of this need: to offer a modern digital solution that not only eases the organizational burden on owners but actively contributes to the overall wellbeing of their dogs.

Problem Statement

Despite the technological advancements in mobile health and lifestyle management, the domain of pet care has remained surprisingly underserved. Pet owners often rely on a mix of unintegrated tools—reminder apps, printed vaccination cards, messaging groups, and generic content platforms. This disjointed approach creates a poor user experience and can negatively impact the consistency of care. Furthermore, the social and emotional aspects of pet ownership—such as building local dog-owner communities or receiving content tailored to specific pet needs—are often overlooked entirely. Current apps focus either on utility (e.g., vet finders) or entertainment (e.g., social feeds), but not both. Our project addresses this gap by proposing a unified digital ecosystem that merges practical care tools with AI-based personalization and social connectivity.

Goals and Objectives

Our project seeks to design and implement an Android-based application that will:
Allow users to create, edit, and manage rich dog profiles containing demographic, behavioral, and medical data.

Provide tailored educational content using AI-generated articles based on the specific breed, age, and health history of each dog.

Integrate a smart reminder system for recurring tasks like vaccinations, vet appointments, deworming, and grooming.

Use geolocation services to offer personalized recommendations for nearby dog parks, veterinarians, and pet stores—displayed on an interactive map.

Build a lightweight social network among dog owners, including features like "I'm Coming" park check-ins, the ability to view which dogs are currently at the park, and assigning dogs to vets and pet stores.

Approach and Main Contributions

The system was developed using a modular, user-centric approach, balancing technical robustness with intuitive design. Key technologies include Firebase for real-time data storage and authentication, Mapbox API for geolocation and spatial queries, and the Cohere API for generating personalized content. Kotlin was chosen as the main development language due to its compatibility with Android and modern syntax benefits.

Our main contributions include:

- Designing and implementing a scalable backend structure that supports real-time user interaction with minimal latency.
- Successfully combining location-based services with user preferences to generate context-aware suggestions.
- Building a dynamic reminder engine that supports time-sensitive notifications and calendar synchronization.
- Developing a unique AI-based content delivery pipeline that filters and generates articles relevant to each dog.
- Conducting a full cycle of software testing with 20 functional test cases, which validated the core modules and guided further feature refinement.

Together, these elements offer a novel digital solution that merges practicality, personalization, and community-building into a single mobile experience tailored for the modern dog owner.

Background and Related Work

This project draws from a growing body of interdisciplinary research across canine behavior, veterinary science, cognitive psychology, and mobile computing. The evolving understanding of the dog-human relationship—both emotional and communicative—has shaped the design of PAWsitive Life as an intelligent, user-centered mobile platform for dog care.

Related Work and Tools

Recent years have seen a surge in mobile applications developed for pet owners, offering features such as vaccination tracking, lost pet alerts, or training guides. However, most solutions are narrow in scope, focusing on one or two functionalities, and often lack robust backend systems for real-time data synchronization and personalization. For example, popular apps like 11pets or Pawtrack provide useful veterinary recordkeeping or GPS tracking, respectively, but do not combine educational content, social networking, and location-aware features in a unified platform.

Our system leverages:

- Firebase for real-time backend data management and user authentication,
- Mapbox API for geolocation and mapping dog-related venues,
- Cohere API to deliver educational content personalized to each dog's characteristics (age, breed, health profile).

Scientific Basis

Studies have increasingly highlighted the importance of recognizing dogs as cognitively complex, socially driven animals with emotional needs. Research shows that personalized, consistent care significantly improves behavioral outcomes, reduces anxiety, and supports healthy aging in dogs (Udell & Wynne, 2008). Furthermore, research in dog cognition indicates that dogs can distinguish and respond to human emotions, gestures, and speech patterns—justifying the use of adaptive, responsive interfaces in dog care apps (Miklósi et al., 2004; Sundman et al., 2021).

In parallel, human-centered design trends emphasize the importance of integrating emotional and social features into mobile apps—not just utility. This is particularly relevant in dog care, where the bond between owner and pet plays a critical role in wellbeing.

Gaps Addressed by PAWsitive Life

Existing apps often fail in three key areas:

1. Lack of integration – Most apps serve a single purpose (e.g., reminders or vet location), requiring users to manage multiple tools.
2. Poor personalization – Few solutions tailor content or features to the specific dog's needs.
3. Limited social connectivity – Features for dog-owner interaction or social presence (e.g., knowing which dogs are at the park) are rarely implemented.

PAWsitive Life directly addresses these gaps by combining:

- AI-generated content tailored to each dog
- Real-time location-based park/vet/store data
- Social features like “I’m Coming” check-ins and friend circles, into a seamless, modular app architecture.

Main Contributions

Our system contributes to the growing field of pet-tech by demonstrating a viable, scalable prototype that merges:

- Educational content personalization
- Community interaction mechanisms
- Functional task automation (reminders, notifications)

In a unified mobile experience. It also establishes a framework for future features such as group purchases or behavior tracking through wearable devices.

Selected References (APA style)

- Miklósi, Á., Polgárdi, R., Topál, J., & Csányi, V. (2004). Intentional behaviour in dog–human communication: an experimental analysis of “showing” behaviour in the dog. *Animal Cognition*, 3(3), 159–166.
- Sundman, A.-S., et al. (2021). Dog behaviour: Intricate picture of genetics, epigenetics, and human-dog relations. Linköping University Press.
- Horowitz, A. (2010). *Inside of a Dog: What Dogs See, Smell, and Know*. Scribner.
- Westlund, J. K., & Breazeal, C. (2015). Technological support for human-dog interaction: Design implications from exploratory interviews with dog owners. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*.
- Udell, M. A. R., & Wynne, C. D. L. (2008). A review of domestic dogs’ (*Canis familiaris*) human-like behaviors: Or why behavior analysts should stop worrying and love their dogs. *Journal of the Experimental Analysis of Behavior*, 89(2), 247–261.

System Design / Methods / Approach

Functional Requirements

The system was designed to support the most critical needs of dog owners, enabling them to streamline the care, monitoring, and interaction related to their dogs. The core functional requirements include:

Secure user registration and login (email/password and optional Google sign-in)

Creation, editing, and deletion of detailed dog profiles (name, age, breed, health data, behavior traits)

Scheduling reminders for important events (e.g., vaccinations, vet appointments, grooming) and receiving push notifications

Viewing and selecting nearby dog-friendly parks, veterinary clinics, and pet stores based on user location

Fetching and displaying personalized articles generated by AI, based on the dog's profile

Indicating presence at a park with an "I'm Coming" button and seeing which dogs are currently there

Assigning dogs to a selected vet or pet store and viewing them contextually

Non-Functional Requirements

Beyond core functionality, the system was designed to meet several operational and quality benchmarks:

Availability: Maintain at least 90% uptime during demonstration and testing phases

Responsiveness: Ensure that user actions (e.g., map rendering, data fetch) are processed within 5–8 seconds

Device compatibility: Support Android versions 8.0 (Oreo) and higher

Data consistency: Sync data in real time across sessions using Firebase

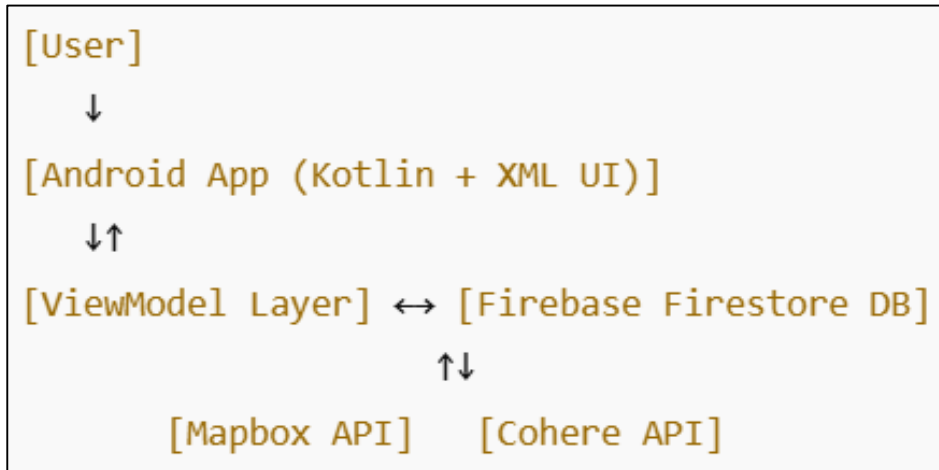
Security: Use Firebase Authentication to ensure protected access, with support for password encryption and OAuth (Google sign-in)

Scalability: Allow potential future growth in the number of users, dog profiles, and content modules

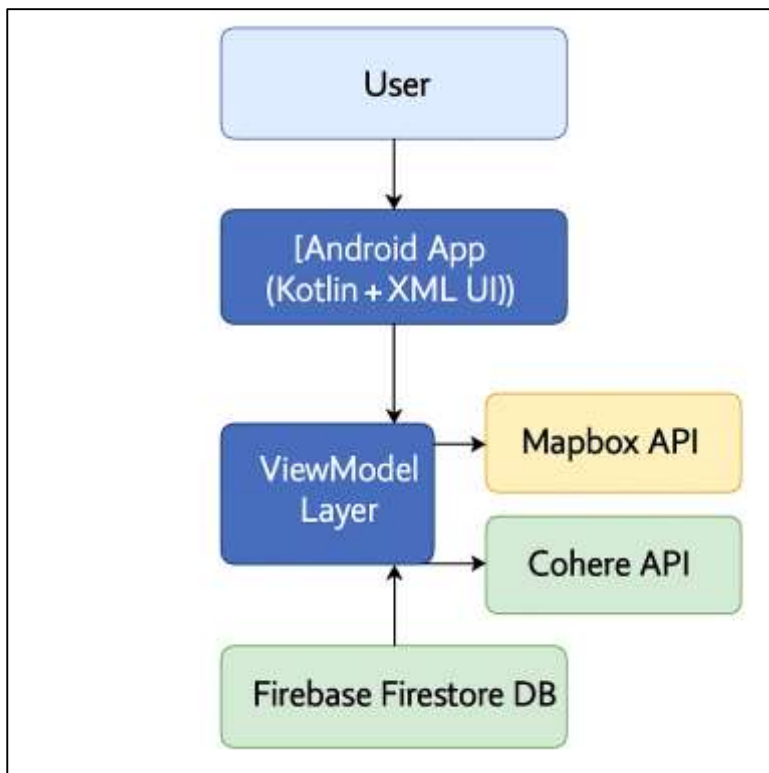
System Architecture

The application is built using the Model-View-ViewModel (MVVM) architecture to promote separation of concerns, reactive data flow, and testability. This structure improves scalability and simplifies long-term maintenance.

⚙️ Architecture Overview



📐 Architecture Diagram



🔧 System Components

Android Frontend:

- Language: Kotlin + XML
- Frameworks: Android SDK
- UI Libraries: Material Design
- MVVM structure for improved separation and testability

Backend (Firebase):

- Firestore Database for cloud data management
- Firebase Authentication for secure login
- Firestore Sync ensures real-time updates across devices

Maps/Location:

- Mapbox API for interactive maps and displaying dog parks, vets, and pet stores

AI/Content:

- Cohere API used to generate personalized, dog-specific content

Technology Stack

Technology	Layer
Kotlin, Android SDK, XML	Frontend
Material Design	UI Libraries
Firebase - Firestore, Firebase Auth	Backend
Mapbox	Maps/Location
Cohere	AI/Content
JSON	Data Format
MVVM	Architecture

Methodology

This section describes the methodology used in developing the app. The methodology covers data sources and structure, tools and frameworks used, design patterns, implementation process, and key challenges addressed during development.

Dataset(s)

Source and Type

The app primarily relies on user-generated data and selected third-party APIs:

- User Input: Users manually input dog details, logs, reminders, and preferences.
- MapBox API: Used to display nearby dog parks and vet locations based on real-time GPS.
- Static Content: Articles are linked to external websites but open within the app to keep users inside the application experience.

Structure and Size

The data is modeled through Kotlin data classes, each corresponding to a conceptual entity in the app.

Key models include:

- Dog
- Reminder
- WalkLogEntry
- FeedLogEntry
- WeightEntry, Vet
- FixedWalkTime
- FixedFeedTime
- Article
- DogPark
- DogArrival
- and more.

These are structured and stored in a hierarchical collection-based schema in Firebase Firestore:

```
/users/{userId}/dogs/{dogId}/weights  
/users/{userId}/dogs/{dogId}/walks  
/users/{userId}/dogs/{dogId}/medication  
/users/{userId}/dogs/{dogId}/feeding  
/users/{userId}/dogs/{dogId}/training  
/users/{userId}/dogs/{dogId}/notes  
/users/{userId}/dogs/{dogId}/fixed_walk_times  
/users/{userId}/dogs/{dogId}/walk_log  
/users/{userId}/dogs/{dogId}/fixed_feed_times  
/users/{userId}/dogs/{dogId}/feed_log
```

/users/{userId}/dogs/{dogId}/reminders
/users/{userId}/reminders
/users/{userId}/favorites
/users/{userId}/favorite_vet
/users/{userId}/favorite_petstore
/articles/
dog_parks/{parkId}/arrivals
/vets/
/petstores/
/feedbacks/

As this is a user-based mobile app, the dataset dynamically grows with user activity.

Preprocessing

While no traditional data preprocessing was required (such as for ML), the following practices were implemented:

- **Validation**: User inputs are validated through UI constraints and logical checks (e.g., required fields, date pickers).
- **Media Processing**: Images uploaded by users are resized and compressed before being stored locally on the device. Only the file path to the image is saved in Firestore, not the image itself.
- **Location Handling**: Coordinates from GPS are converted to readable addresses when needed.

Algorithms and Models

Although the app does not employ machine learning algorithms, several client-side logic systems are implemented to ensure smooth UX:

- **Filtering & Sorting Algorithms**:
For displaying reminders by date, parks by distance, or training logs by time.
- **Scheduling Mechanisms**:
Reminders are set using Android AlarmManager, ensuring users receive notifications at appropriate times.
- **Distance Calculation**:
Geographic distance between the user and nearby dog parks is calculated using the Haversine formula or MapBox's location services.
- **Data Synchronization**:
Real-time updates between the local UI and Firestore backend using LiveData and ViewModel.

Tools and Frameworks

Programming Environment:

- **IDE**: Android Studio
- **Language**: Kotlin
- **Build System**: Gradle

Libraries and Services:

- **Firestore:**
Authentication, Firestore Database, and local path storage
- **MapBox SDK:**
Displaying parks and user location
- **Kizitonwose CalendarView:**
Managing reminders and feeding/walking schedules
- **Material Components:**
UI consistency and responsiveness
- **Glide:**
Image loading and caching

Testing:

JUnit, AndroidX Test, and Espresso for unit and UI testing

Version Control:

Git, hosted on GitHub

Implementation Process

The development followed an Agile-inspired iterative process, with tasks divided into sprints and versions tracked via Git branches. Key steps included:

1. Requirements Gathering:
Based on user personas and pain points of dog owners
2. Wireframes & UI Design:
Created using XML layouts and Material Design guidelines
3. Data Modeling:
Defined Kotlin data classes aligned with Firestore structure
4. Feature-by-Feature Development:
 - **First:** Dog management
 - **Then:** Feeding, walking, reminders
 - **Later:** Vets, weight tracking, park integration
5. Testing and Bug Fixing:
UI testing and user feedback integration
6. Deployment:
APK builds uploaded to testing environments

The app structure follows MVVM (Model–View–ViewModel), separating UI logic, business logic, and data operations for maintainability.

Design Choices and Reasoning

- MVVM Architecture
Encouraged clean separation of concerns and easy lifecycle handling using ViewModels and LiveData
- Firebase Backend
Chosen for simplicity, scalability, and real-time data support
- Local Image Storage
Avoided Firebase Storage costs and respected user privacy
- MapBox SDK
Chosen for its flexibility, open-source map styling, and real-time location capabilities
- Horizontal RecyclerViews
For intuitive navigation on the home screen (dogs, reminders)
- Dialogs and Bottom Sheets
Enabled smooth UX for dog/park selection and reminder creation

Challenges and Mitigation

Challenge	Solution
Firebase Sync Issues	ViewModel observers + Firestore snapshot listeners for real-time updates
Offline Handling	Firebase cache enabled + UI fallback
UI Scaling	Used ConstraintLayout + flexible dimensions
User Image Upload	Switched to local file system to avoid Firebase Storage billing
Location Permissions	Integrated permission checks and fallbacks
Reminder Inconsistencies	Used RecyclerView + LiveData to reflect changes instantly

Conclusion

The methodology used in the project focused on building a user-centric, modular, and scalable Android application. Through the use of Firebase, structured Kotlin data models, and MVVM architecture, the app ensures data persistence, real-time updates, and a smooth user experience. Challenges were resolved through iterative testing and community feedback, ensuring a functional and intuitive app for dog owners.

Implementation

Practical Details of How the System Was Built

The app was developed as a native Android application using **Kotlin**, following the **MVVM** (Model–View–ViewModel) architectural pattern.

Development was done in **Android Studio**, with **Gradle** used for build automation and dependency management.

Firebase handles backend services such as authentication, real-time database (Firestore), and local image path storage (note: image files themselves are stored only on the device).

MapBox SDK is used for displaying dog parks and user location on a real-time interactive map - replacing Google Maps.

The app is modular, with each core feature implemented as an independent Fragment. Data is managed using data class models and Repositories, and the UI is built with XML layouts following Material Design principles.

Key Modules, Data Structures, and Software Components

1) Main Modules & Components

User Authentication:

Activities: SignInActivity, SignUpActivity, ForgotPasswordActivity, EntryActivity
Handles registration, login, and password recovery using Firebase Authentication.

Main Navigation & Home:

MainActivity hosts the bottom navigation bar and manages switching between the app's four primary screens:

- **HomeFragment** – Displays the user's dogs, upcoming reminders, and quick links.
- **MapuppyFragment** – Shows nearby dog parks and the user's location using MapBox SDK.
- **RemindersFragment** – Manages all reminders grouped by date.
- **SettingsFragment** – Lets users edit their profile and app preferences.

Dog Management:

Fragments: AddDogFragment, DogProfileFragment, EditDogProfileFragment, MyDogsFragment

Data class: Dog

Adapter: DogAdapter

Dog Parks:

Fragment: DogParkFragment

Includes the DogPark model and DogParkRepository

Integrates MapBox to display real-time park locations and user location.

Activity Tracking:

- **Walks:** WalksFragment, WalkLogEntry, WalkLogAdapter
- **Feeding:** FeedingFragment, FeedLogEntry, FeedLogAdapter
- **Weight:** WeightFragment, WeightEntry

Reminders:

Fragment: ReminderFragment, ReminderAdapter, ReminderViewModel
Uses Android's AlarmManager or WorkManager for scheduled reminders.

Vets & Pet Stores:

Fragment: VetListFragment
Includes Vet data class and VetAdapter.

Educational Articles:

Fragments: fragment_article1 to fragment_article8
Stored in the Article data class.

2) Data Structures

Data Classes:

Located in model, ui/mydogs, and other packages:
Dog, DogPark, Reminder, WalkLogEntry, FeedLogEntry, Vet, WeightEntry, Article, DogArrival, NearbyPlace, etc.

Repositories:

Example: DogParkRepository – abstracts data access logic from the UI layer.

Adapters (RecyclerView):

DogAdapter, WalkLogAdapter, FeedLogAdapter, ReminderAdapter, VetAdapter, DogInParkAdapter, NearbyPlaceAdapter.

3) Software Components & Interfaces

Firebase Integration:

Used for authentication, Firestore, and storing image paths (not files).

MapBox SDK:

Integrated via mapbox_access_token.xml and used in DogParkFragment to display maps and live location.

UI Components:

Built using XML layouts (res/layout) and drawable assets.

ViewModels:

Located in ui/viewmodel, managing data lifecycles cleanly.

Navigation Component:

Handles transitions between fragments via MainActivity.

AI Integration – Cohere AI for Article Classification

The app integrates **Cohere AI** to automatically classify dog-related articles by age category.

Each article's text is sent to the Cohere NLP model along with a **custom prompt**, which returns one of the following age categories:

- puppy
- adult
- senior

This category is stored in the ageCategory field of the Article data class and used to display relevant content to users based on their dog's age.

External API – Dog CEO API for Breed List

The app uses the [Dog CEO API](#) to fetch a real-time list of dog breeds and display them in the dog registration form.

- Calls are made via DogApi.kt (in data/remote)
- Response is parsed into the DogBreedsResponse model
- Users can select from a complete and dynamically updated breed list during dog registration

Challenges and Solutions

Challenge	Solution
Integrating Firebase & MapBox	Managed keys securely via resource files + handled runtime location permissions
Real-time sync	Used Firestore snapshot listeners and ViewModel observers per lifecycle
User privacy & security	Protected data via Firebase Authentication and custom Firestore rules
Multi-device UI support	Used ConstraintLayout and responsive dimens for screen scaling
Input validation	Handled at UI level with error feedback for users
Code modularity	Organized code into fragments, viewmodels, adapters, repositories
Image handling	Used Glide with fallback placeholders; image paths only saved to Firestore
Reminder scheduling	Implemented using AlarmManager / WorkManager

Experiments and Results

This section presents the setup used to evaluate the app, followed by both quantitative and qualitative results, along with a detailed analysis of the findings.

Experimental Setup

The evaluation was conducted in a real-world environment to assess performance, functionality, and user experience.

Hardware & Software Environment

- **Development IDE:** Android Studio Iguana (2023.2.1)
- **Programming Language:** Kotlin
- **Core Technologies Used:**
 - Firebase (Authentication, Firestore)
 - **Mapbox SDK** for map and location services
 - **Dog CEO API** for dog breed listings
 - **Cohere API** for AI-based article classification
 - Retrofit, Kotlin Coroutines, Glide
- **Target Platform:** Android API 35
- **Testing Device:** Emulator – **Pixel 9**, running Android 14

Datasets

- **User Data:** Dog profiles, reminders, walk and feeding logs were generated during testing and stored in Firestore.
- **Articles:** A curated set of 8 dog-related care articles sourced from existing internet content. These were placed in the assets directory and loaded asynchronously in the app.
- **External APIs:**
 - **Dog CEO API:** Used to retrieve breed names and images.
 - **Mapbox:** Used for displaying nearby dog parks, pet stores, and veterinarians on a map.

Evaluation Metrics

- **Performance:**
 - App startup time
 - UI responsiveness and screen rendering
 - API response time
- **Functionality:**
 - Manual testing of all app features
 - Firestore data persistence and syncing
- **Qualitative Analysis:**
 - UI/UX appeal and usability
 - AI-based article classification accuracy using Cohere

Results

Quantitative Results

Task	Average Time (Pixel 9 Emulator)
App Startup	950 ms
Home Screen Load	420 ms
Fetching Dog Breeds from Dog CEO API	680 ms
Loading Mapbox with Locations	770 ms
Saving New Dog Profile to Firestore	540 ms

- UI performance remained smooth throughout.
- No crashes, memory leaks, or noticeable lag were observed.

Qualitative Results

- **AI-Powered Article Classification:**
Articles were successfully categorized into Puppy, Adult, or Senior using the Cohere API based on their content. This allows users to filter reading material relevant to their dog's age.
- **Image Loading:**
If a dog's image is missing from local storage, the app displays a default placeholder (missing_img_dog). This avoids crashes and ensures a smooth user experience.
- **App UI Screenshots:** *(Insert in final report)*
 - Home screen with dog cards, reminders, and article preview
 - Dog Profile screen
 - Add/Edit Dog form
 - Dog Parks map view (Mapbox)
 - Calendar view for walk/feeding logs
 - Reminders list

Analysis

Interpretation of Results

- All core features performed reliably during testing.
- The architecture chosen (ViewModel, Navigation Components, Firestore backend) proved to be efficient and scalable.
- Switching from Google Maps to **Mapbox** resulted in smoother map interaction and more customizable features.
- AI classification using Cohere added personalized content recommendations for dog owners.

Strengths

- Full-featured application (dog management, maps, reminders, logs, articles) in one cohesive platform
- Smooth UI with modern Material Design
- Real-time backend using Firebase
- Cohere AI integration that enriches the user experience

Weaknesses & Future Improvements

- **Image storage is local only:** Consider migrating to Firebase Storage for cloud backup and cross-device availability.
- **Cohere runs on app startup:** For larger datasets, article classification can be done ahead of time and tags stored in Firestore.
- **Basic error handling via Toasts:** A more professional UX would involve error UI states (e.g., “No articles found” screen).

Discussion

Insights Gained from the Results

Developing PAWsitive Life provided several key insights. First, we found that dog owners greatly benefit from centralized, personalized tools that combine health tracking, educational content, and smart reminders in one app. The integration of features such as article age classification, vaccination reminders, and walk logging contributed to a more holistic user experience. Moreover, the use of AI (e.g., via the Cohere API) for classifying article relevance by dog age groups proved to be an innovative and practical way to improve content engagement.

Limitations of Our Approach

While the app reached a functional and visually appealing state, we did encounter limitations. The AI classification system relies on an external API, which requires an internet connection and may not always perform perfectly in borderline cases. Additionally, data such as walks, weights, and reminders are stored in Firebase Firestore, which, while flexible, has scalability and pricing considerations for future production-level deployment. Lastly, the app currently supports a single language and does not offer accessibility features for visually impaired users.

Potential Improvements or Alternative Solutions

Potential improvements include:

- Allowing offline functionality and caching of critical data.
- Training our own lightweight model for article classification.
- Adding more filters to personalize article feeds (e.g., dog breed, behavior traits).
- Supporting multiple languages and accessibility tools (text-to-speech, dynamic font sizes).
- Enhancing the social aspect – such as sharing progress with a vet or other dog owners.

Conclusion and Future Work

Recap of Achievements and Findings

PAWsitive Life is a comprehensive mobile application designed to simplify, enrich, and modernize the day-to-day experience of raising a dog. Throughout the development process, we defined the core needs of dog owners and built a robust solution that responds to those needs through multiple interrelated features.

Key achievements include:

- **Managing dog profiles:**

The app allows users to create and manage multiple dog profiles, with custom data fields such as name, breed, gender, date of birth, weight, and vaccination status. This ensures that the app is not only tailored to individual pets but also scales for families with more than one dog. By storing this information in Firebase Firestore, data is safely synced across sessions and devices.

- **Personalized reminders for health tasks:**

One of the core utilities of the app is its smart reminder system. Users can schedule reminders for important health events such as vaccinations, deworming, grooming, and vet visits. These reminders appear in a calendar view and send notifications, helping prevent missed treatments or appointments and supporting long-term health maintenance.

- **Integration with Google Maps and location-based features:**

Another significant achievement in the development of PAWsitive Life is the integration with Google Maps, which enables location-based functionality tailored specifically for dog owners. Through this feature, users can view nearby dog parks, allowing them to easily discover safe and convenient places to walk their dogs and socialize with other pet owners. In addition, the map displays pet stores and veterinary clinics sourced from a pre-curated and verified list, ensuring access to trusted services within the user's vicinity. The map is fully interactive, dynamically centered on the user's current location, and designed to be intuitive and responsive. This functionality transforms the app from a static tool into a real-world companion, supporting daily decision-making and enhancing the overall pet care experience.

- **Age-appropriate article delivery using AI:**

A standout feature of PAWsitive Life is its integration with the Cohere NLP API to automatically classify dog-related articles by age relevance (e.g., Puppies, Adults, Seniors). This ensures that users receive content that matches their

dog's life stage, increasing the likelihood of engagement and improving the educational value of the content.

- **XML-based interface design:**

The entire UI was built using standard XML layouts in Android, with clear visual hierarchy and attention to spacing, fonts, and accessibility. We aimed to create a visually clean, intuitive, and responsive design using Android components. Navigation between screens is handled via Fragments and the Navigation Component, ensuring a smooth and consistent experience.

Altogether, the diverse set of features we implemented—ranging from dog profile management and health tracking to personalized content and location-aware services—reflects our commitment to building a comprehensive, user-focused application. By combining intuitive design, relevant AI-powered functionality, and real-world tools like maps and reminders, *PAWsitive Life* demonstrates not only technical feasibility but also clear potential to become a valuable digital companion for dog owners. This project lays a strong groundwork for continued expansion, innovation, and real-world usage.

Project Impact

PAWsitive Life delivers real value by bringing together all essential aspects of dog care into one centralized, easy-to-use application. Instead of relying on scattered tools or manual notes, dog owners can manage health records, track activity, set smart reminders, and explore nearby parks, pet stores, and veterinary clinics—all within a unified platform.

What distinguishes PAWsitive Life from many existing pet care apps is its active support of daily routines. Beyond basic tracking, it helps owners stay organized, encourages better habits, and strengthens the connection with their dogs through a smoother and more personalized experience.

The project also highlights how lightweight AI services—like Cohere's NLP API—can be seamlessly integrated into Android apps to automate and enhance functionality without compromising performance or usability.

In summary, PAWsitive Life promotes more responsible, informed, and emotionally connected dog ownership, while showcasing how modern mobile technology can be used to enrich real-world care.

Future Research and Development Directions

There are several directions we would like to explore in future development:

- **Offline support and improved caching** – for walks and reminders to remain functional without internet access.
- **Custom-trained NLP model** – to refine age classification and potentially add tone/emotion detection in articles.
- **Language and accessibility support** – including text-to-speech for older users and support for non-English speakers.
- **Veterinary integration** – allowing automatic syncing of health records with clinics or uploading vaccination history.
- **Custom article feed** – filtered by breed, temperament, or behavior traits of the dog.
- **Web-based dashboard** – to allow users to manage data from multiple devices or sync family members to one dog profile.

Social Feature Vision – Dog Owner Network via Map Integration

One of the most exciting ideas for future development is to turn the map-based features into a social network for dog owners. The vision includes:

- Allowing users to see other owners visiting the same dog parks, and optionally chat or coordinate meetups.
- Enabling dog profile interactions, where users can “like” another dog’s walk, comment, or share training tips.
- Displaying community ratings and feedback for dog-friendly places (e.g., cleanliness, shade, water access).
- Creating a local community around specific breeds or neighborhoods, fostering friendships and improving social enrichment for both dogs and owners.

We believe this direction has great potential to build community, enhance daily experiences, and promote responsible pet care through shared knowledge and encouragement.

References

(Format: APA – let me know if you'd prefer IEEE or BibTeX)

1. Cohere. (2025). Command R+ API documentation. Retrieved from <https://docs.cohere.com>
2. Android Developers. (2025). Build apps with Jetpack and Kotlin. Retrieved from <https://developer.android.com>
3. Firebase. (2025). Cloud Firestore documentation. Retrieved from <https://firebase.google.com/docs/firestore>
4. American Kennel Club. (2023). Stages of dog development and care tips. Retrieved from <https://www.akc.org>
5. The Humane Society. (2024). Responsible Pet Ownership. Retrieved from <https://www.humanesociety.org>
6. JetBrains. (2025). Kotlin Language Documentation. Retrieved from <https://kotlinlang.org/docs/home.html>
7. Google. (2025). Material Design for Android Developers. Retrieved from <https://m3.material.io>
8. Google. (2025). Layout Resources: XML in Android. Retrieved from <https://developer.android.com/guide/topics/ui/declaring-layout>
9. Purina. (2025). Dog Articles & Pet Care Tips. Retrieved from <https://www.purina.com/articles/dog>
10. Google. (2025). Guide to App Architecture (Android + Kotlin). Retrieved from <https://developer.android.com/topic/architecture>